

JAVA INTERVIEW PREPARATION

CHAPTER 63

THREAD EXHAUSTION AND
POOL STARVATION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Diagnosing Thread Exhaustion, Pool Starvation, and Request Hangs

Senior / Lead Java Developer Reading Chapter

When a Java service stops responding, the immediate explanation is often "all threads are stuck." That phrase can describe several different failures: a server worker pool is full, an executor queue is saturated, threads are blocked on a connection pool, tasks wait on other tasks in the same pool, a lock owner cannot progress, platform threads were created without bound, or virtual threads are pinned behind scarce carriers.

The diagnosis must identify which population cannot progress, what resource it waits for, and who owns that resource.

```
incoming work -> server admission -> application executor
               -> connection pool -> database/downstream
               -> locks/coordination -> completion
```

Exhaustion at any stage can make every upstream stage appear stuck.

Define the symptom and scope

Collect:

- request throughput, p95/p99 latency, timeouts, and rejection rate;
- in-flight requests and server busy/max threads;
- executor active threads, pool size, queue depth, rejection, and task age;
- database and HTTP connection active/idle/pending counts;
- live, peak, and newly created platform threads;
- virtual-thread counts and pinning events where used;
- CPU, container throttling, GC, heap, and file descriptors;
- affected route, version, instance, zone, and start time.

A service with low CPU and all request threads waiting has a different problem from a service with every thread runnable and CPU saturated. A single bad instance suggests local skew or leak; fleet-wide behavior suggests shared dependency, workload, or rollout.

Protect service while preserving thread state

Mitigation may shed new work, disable expensive routes, pause batch producers, break retry loops, route around a bad dependency, or replace instances gradually. Before restarting the only affected process, capture repeated thread evidence.

```
thread dump 1 at T0
thread dump 2 at T0 + 5 seconds
thread dump 3 at T0 + 10 seconds
```

Also save pool metrics and traffic state for the same interval. A stack is meaningful only with the resource population around it.

Traditional platform-thread dumps can be captured through approved JDK mechanisms such as `jcmd Thread.print` or platform signals, depending on environment. For Java 21 virtual threads, `jcmd <pid> Thread.dump_to_file` can write text or JSON containing platform and virtual threads. Validate commands

and impact for the deployed JDK.

Interpret thread states correctly

Java thread states are clues:

- **RUNNABLE**: executing or eligible to execute; may include native I/O details depending on stack;
- **BLOCKED**: waiting to enter a **synchronized** monitor;
- **WAITING**: waiting indefinitely for notification, join, park, latch, or another condition;
- **TIMED_WAITING**: similar, with a timeout;
- **NEW** or **TERMINATED**: not active work.

Many **WAITING** threads can be healthy idle workers. Many **RUNNABLE** threads do not prove thread exhaustion; they may indicate CPU saturation. A **BLOCKED** population is significant when one owner and one monitor dominate.

Read the stack, lock information, thread name, and repeated progression. Group identical or similar stacks to find the largest waiting population.

Build a thread-state census

Instead of reading hundreds of dumps linearly, classify by executor and top meaningful frame:

```
120 http-worker threads
  90 waiting HikariPool.getConnection
  20 reading downstream response
   8 serializing
   2 idle
```

Root bottleneck is database connection ownership, not HTTP worker count.

Thread names should identify purpose: **http-***, **payment-client-***, **batch-***, **scheduler-***. Default anonymous pool names slow incidents and can mix unrelated executors.

For each dominant group, ask what wakes it, whether a timeout exists, and which thread or external system owns the awaited progress.

Server worker-pool saturation

In a platform-thread request model, the server assigns requests to a bounded worker pool. If every worker blocks on slow I/O, new requests wait in the accept or server queue until timeout or rejection.

```
200 server workers
200 blocked on downstream without effective timeout
new requests queue even for endpoints that do not need that downstream
```

Increasing workers may provide temporary capacity but can create more database connections, remote calls, memory, and context switching. Isolate optional dependencies with bulkheads, enforce deadlines, and ensure a small slow route cannot occupy the entire shared pool.

Measure active workers, maximum, queued requests, connection acceptance, response completion, and slow-client write time. A worker may remain occupied while streaming to a slow client after business logic completes.

Executor saturation and queue behavior

An executor has threads, a work queue, and a rejection policy. The interaction matters more than the thread count alone.

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    16, 16,
    0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(500),
    namedThreadFactory("invoice-worker-"),
    new ThreadPoolExecutor.AbortPolicy());
```

The correct values follow measured service time, arrival rate, CPU, memory, and downstream capacity. A bounded queue makes overload visible. `CallerRunsPolicy` can propagate backpressure in some synchronous producer designs, but it can also block an event loop, request thread, or scheduler and spread starvation. Choose rejection deliberately.

Monitor queue age as well as depth. A stable queue of old tasks is worse than a brief burst. Attach deadlines to tasks so expired work does not execute after the caller has given up.

Capacity arithmetic for executors

Pool sizing begins with work type and measured service time. For CPU-bound tasks, useful parallelism is near available cores, adjusted through benchmark evidence. For blocking work, more concurrency can hide waiting, but downstream capacity sets the real ceiling.

Little's Law provides a consistency check:

```
concurrency ~= arrival rate * average time in system

300 tasks/s * 0.2 s ~= 60 concurrent tasks
```

If only 30 connections exist, configuring 200 worker threads creates 170 potential waiters, not 200 units of throughput. If the arrival rate exceeds sustainable completion, every finite queue eventually fills.

Choose queue capacity from acceptable burst duration and memory cost:

```
temporary excess = arrival rate - completion rate
queue required   ~= temporary excess * tolerated burst seconds
```

This is not permission to queue until the client timeout. Admission should reject before queued age makes useful completion impossible. Estimate retained bytes per queued task, including captured payload and context.

Rejection is part of the contract

A bounded executor must define what happens when full. `AbortPolicy` fails explicitly; caller-runs slows the producer; discard policies lose work and are unsafe unless the work is intentionally optional and measured.

For synchronous APIs, translate overload into a controlled 503 or other contractually suitable response before expensive work. For asynchronous durable jobs, reject publication, delay consumption, or retain work in a broker with explicit capacity and age limits.

Reserve capacity for health, cancellation, recovery, and critical operations. If the same pool handles user work and timeout cleanup, saturation can prevent the mechanism intended to release resources.

Alert on rejection and queue age, but treat rejection during a deliberate overload test as successful protection. The failure is admitting work that cannot complete while reporting no rejections.

Unbounded queues hide pool exhaustion

Many executor factories use unbounded queues or pools with surprising behavior. With an unbounded queue, a configured maximum thread count may never be reached because work is queued after core threads are busy. Latency and memory grow while the dashboard shows no rejections.

Every queued lambda can retain request payload, security context, tracing state, buffers, and futures. Pool starvation becomes a memory incident.

Prefer explicit executors with documented ownership, boundedness, rejection, shutdown, and observability. Do not create a new executor per request or leave pools unclosed during redeployment.

Connection-pool starvation looks like thread starvation

Threads often wait because a downstream connection pool is exhausted. Common causes include slow SQL, long transactions, remote calls while holding a database connection, leaked connections, undersized pool, or a fleet whose total connections overwhelm the database.

```
request threads -> wait for 20 DB connections
20 connection owners -> wait on slow lock or remote call
```

Find the connection owners, not only the waiters. Pool metrics should expose active, idle, pending, maximum, acquisition duration, and timeout. Database evidence should identify long transactions, blockers, and query duration.

Increasing the pool can worsen the database bottleneck. Calculate the total across all application replicas, batch jobs, admin tools, and failover capacity.

HTTP connection pools create the same pattern. Separate pool acquisition from connect, TLS, request, and response-body read. Reuse configured clients instead of creating one client and pool per request.

Thread-starvation deadlock

A thread-starvation deadlock occurs when tasks in a bounded pool wait for other tasks that must run on the same fully occupied pool.

```
ExecutorService pool = Executors.newFixedThreadPool(8);

Future<Result> outer = pool.submit(() -> {
    Future<Result> inner = pool.submit(this::loadDependency);
    return inner.get(); // all 8 outer tasks can wait for queued inner tasks
});
```

No Java monitor cycle may be detected, yet progress is impossible. Repeated dumps show every worker waiting on futures while the required tasks remain queued.

Avoid blocking joins inside the same constrained pool. Compose asynchronously, run dependent stages inline where safe, separate pools by workload, or reserve capacity. Pool separation must reflect actual dependency direction; arbitrary extra pools can simply move the deadlock.

CompletableFuture and common-pool starvation

Async methods without an explicit executor may use the common `ForkJoinPool`. Blocking I/O, `join`, or long CPU work there can delay unrelated tasks in the JVM.

Use explicit executors for blocking operations and understand which thread executes each continuation. `thenApply` may execute on the thread completing the previous stage, while async variants use an executor.

Hidden execution placement can block a request thread or critical callback.

Timeouts on a future do not automatically stop the underlying operation. Orphaned work can continue occupying threads and connections after callers fail, creating a feedback loop.

Lock contention and monitor ownership

Many **BLOCKED** threads pointing to one monitor indicate serialized access. Identify the owner stack and how long it remains in the critical section.

Common causes include:

- network or database I/O inside **synchronized** code;
- coarse cache refresh lock;
- class initialization waiting on slow work;
- logging or callback execution under a registry lock;
- inconsistent lock order.

Move slow work outside the critical section, reduce lock scope, partition state, or use a coordination primitive that matches the invariant. Replacing **synchronized** with **ReentrantLock** does not help if the same slow work remains serialized.

Deadlock versus hang

A monitor deadlock is a cycle: thread A owns lock 1 and waits for lock 2, while thread B owns lock 2 and waits for lock 1. Traditional JVM thread dumps can perform deadlock detection and report involved monitors.

```
thread A: owns account-1 -> waits account-2
thread B: owns account-2 -> waits account-1
```

Not every hang is a detected deadlock. Threads can wait forever on a latch that is never counted down, a lost notification, a future whose producer failed, an external call without timeout, or starvation in an executor queue. Understand the condition required for wake-up and locate its owner.

Use consistent lock ordering and avoid calling unknown code while holding locks. Timeouts can prevent infinite waiting but do not repair broken invariants; timed-out operations may leave partial state requiring cleanup.

External calls without effective deadlines

One downstream outage can occupy every thread if connect, pool acquisition, headers, or body read lacks a bounded timeout. A total client timeout is not enough if it does not interrupt the blocked phase or cancel underlying work.

Propagate a request deadline and allocate remaining time to each attempt. Retry only transient, idempotent operations within a small budget. Bulkhead the dependency so it cannot consume all server capacity.

Avoid remote calls while holding database transactions, locks, or scarce connections. This couples two failure domains and turns remote slowness into local starvation.

Platform-thread leaks and native limits

If live platform-thread count rises continuously, find who creates them and why they do not terminate. Causes include new executors per request, unclosed schedulers, library clients with separate pools, stuck tasks, and repeated application-context creation.

Platform threads consume native stack and OS resources. Failure may appear as `OutOfMemoryError: unable to create native thread`, process-limit errors, or container OOM. Reducing stack size can delay failure but does not fix runaway creation.

Record peak and creation rate, group names, and compare dumps over time. Ensure executors have lifecycle ownership and graceful shutdown with bounded wait.

Scheduled-executor starvation

A single-thread scheduler that runs a long task delays every later task. Fixed-rate scheduling can create catch-up behavior; exceptions may cancel future executions depending on API usage and handling.

Do not perform long remote work directly on a coordination scheduler. Use it to trigger bounded work on an appropriate executor, with overlap prevention and ownership. Monitor scheduled delay and last successful completion.

Clock changes and time-zone mistakes can create bursts of scheduled work. Use explicit business time and idempotent job identity.

Virtual threads change the symptom, not the resource limit

Virtual threads make blocking concurrency inexpensive because they normally unmount from carrier threads during supported blocking operations. They should not be pooled to limit concurrency. Use semaphores or resource pools to protect scarce downstreams.

Large numbers of virtual threads can all wait for a twenty-connection database pool. This is not thread exhaustion in the traditional sense, but it is still admission and memory pressure. Measure in-flight virtual tasks, wait reason, age, and downstream permits.

A virtual thread can remain pinned to its carrier while blocking inside `synchronized` code or native/foreign calls. Frequent long pinning can reduce available carriers and harm scalability. JFR can record `jdk.VirtualThreadPinned`, and Java 21 thread dumps can show virtual-thread structures.

Do not replace every `synchronized` block. Short nonblocking critical sections are normally fine. Correct frequent long-lived pinning on hot paths, especially when I/O occurs while pinned.

CPU saturation can imitate exhaustion

If many threads are `RUNNABLE` and CPU or container quota is saturated, they wait for scheduling rather than a Java lock. Adding threads worsens context switching. Use the high-CPU workflow, sampling profiles, and CPU-per-operation metrics.

GC pauses and JVM operations can stop application progress across many threads. Correlate pause timestamps with request hangs before attributing them to executors.

A slow or blocked process with low CPU points more strongly toward waits, but low reported CPU can also result from container throttling or measurement units. Confirm host and cgroup views.

Repeated dumps reveal progress

Compare thread groups across dumps:

- same owner in same slow stack: likely blocked external work or loop;
- same waiters and unchanged queue: likely starvation or lost signal;
- changing stacks with slow progress: capacity saturation;
- growing thread count: creation leak or stuck lifecycle;

- monitor cycle reported: deadlock;
- many identical timed waits: timeout or retry wave.

Thread dumps are not perfectly synchronized pictures of the whole system, especially virtual-thread dumps designed without stopping the VM. Combine them with pool and trace timelines.

Recovery storms and stale queued work

When a dependency returns, thousands of waiting or queued tasks may resume together. Connection pools reconnect, caches reload, retries fire, and expired requests compete with new traffic. The system can fail again even though the original dependency is healthy.

Tag work with an absolute deadline or creation time. Before execution or retry, reject work that can no longer produce a useful client or business outcome. Cancellation should propagate to downstream calls where supported, and ambiguous writes require reconciliation rather than blind replay.

Ramp concurrency gradually after recovery. Prioritize current critical traffic over obsolete bulk work, and keep retry jitter active. Measure net drain rate: if completion only matches new arrival, the backlog never clears.

$$\text{backlog drain rate} = \text{recovery completion rate} - \text{new arrival rate}$$

Run recovery tests, not only steady-state saturation tests. A system that survives dependency failure but cannot drain safely is not resilient.

A disciplined diagnosis sequence

1. confirm latency/throughput and CPU state
2. identify saturated server, executor, connection, or permit pool
3. capture repeated dumps and group dominant stacks
4. locate resource owners and wake-up conditions
5. check queue age, deadlines, retries, and orphaned work
6. distinguish lock cycle, starvation dependency, external wait, and CPU
7. mitigate admission and dependency amplification
8. correct ownership/bounds and verify recovery under load

Start from the queue or pool whose waiters correlate with the symptom. Reading random stacks without a population model is slow and error-prone.

Safe mitigation

Reject or shed new low-priority work before queues become unrecoverable. Pause producers and batch jobs, disable optional fan-out, open a circuit where safe, reduce retry attempts, or route around a failed dependency. Restart affected instances gradually if required, preserving one evidence set.

Increasing thread, queue, or connection limits is safe only when the downstream has spare capacity and the service has memory and CPU headroom. Otherwise it transfers the incident.

After the dependency recovers, control catch-up. Old queued tasks may no longer be useful and can create a recovery storm. Honor deadlines and discard expired work with correct business reconciliation.

Verify the correction

Reproduce the failure condition: slow dependency, exhausted connection pool, nested task submission, lock contention, virtual-thread pinning, or thread leak. Under realistic arrival load, verify:

- bounded in-flight work and queue depth;
- queue and task age return to baseline;

- timeouts and rejection occur before global exhaustion;
- critical endpoints retain reserved capacity;
- thread and connection counts plateau;
- no orphaned work continues after caller timeout;
- throughput and p99 recover after dependency restoration.

Add alerts on pending acquisition, queue age, rejection, thread growth, pinning, and recovery time. A high thread-count alert alone is inappropriate for virtual-thread applications.

Common senior-level traps

"Increase the server thread pool." More threads may only create more blocked database or downstream work.

"All waiting threads are stuck." Idle workers wait normally; identify dominant stacks, owners, and progress across dumps.

"No deadlock was detected, so there is no deadlock." Pool starvation and lost signals can halt progress without a monitor cycle.

"The timeout freed the thread and dependency." Underlying work may continue and consume capacity unless cancellation is effective.

"Virtual threads eliminate exhaustion." They remove platform-thread scarcity for blocking code, not connections, CPU, locks, permits, or memory.

"Use a larger unbounded queue to absorb bursts." It converts overload into latency, retained objects, and slow recovery.

A strong interview answer

I begin with throughput, latency, in-flight work, CPU, server workers, executor queues, connection-pool waiters, and thread-count trends. I preserve repeated thread dumps and matching pool metrics, group stacks by named executor and wait point, and locate the resource owner and wake-up condition. I distinguish server saturation, unbounded queues, connection starvation, tasks waiting on the same pool, lock cycles, external calls without deadlines, platform-thread leaks, CPU saturation, and virtual-thread pinning. Mitigation sheds or bounds new work and stops retry amplification rather than blindly increasing pools. The correction uses explicit executor ownership, bounded queues, deadlines, cancellation, bulkheads, resource permits, and tested recovery. I verify queue age, thread and connection plateaus, p99 latency, rejection behavior, and catch-up after the dependency recovers.

Chapter summary

Thread exhaustion is usually resource-ownership failure expressed through threads. Repeated dumps show where populations wait; pool metrics show the bounded resource; traces and dependency evidence explain why owners do not release it. Predictable systems bound admission and make waiting, rejection, cancellation, and recovery explicit.

Revision Notes

- Identify which thread, executor, connection, or permit population cannot progress.
- Capture repeated thread dumps with matching pool and traffic metrics.
- `WAITING` can be healthy idle state; `RUNNABLE` can indicate CPU pressure.
- Group stacks by named executor and meaningful wait point.
- Find resource owners, not only the waiting threads.
- Increasing server threads can transfer saturation to downstream systems.
- Executor behavior depends on threads, queue type, and rejection policy.
- Unbounded queues hide overload as latency and retained memory.
- Measure queue age as well as queue depth.
- Connection-pool starvation often appears as request-thread starvation.
- Calculate database connections across all replicas and jobs.
- Tasks waiting for tasks in the same full pool create starvation deadlock.
- Use explicit executors for blocking `CompletableFuture` stages.
- Timeouts do not automatically cancel underlying work.
- A detected monitor cycle is only one kind of hang.
- External calls need bounded phases, total deadlines, retry budgets, and bulkheads.
- Platform-thread growth consumes native stacks and OS resources.
- Virtual threads should not be pooled to limit concurrency; use semaphores or resource pools.
- Investigate frequent long virtual-thread pinning on hot blocking paths.
- Verify bounded queues, plateaus, rejection, cancellation, and recovery under load.